

Alice in Action with Java

Chapter 9 *Methods*

Objectives

- Use Math methods
- Use string methods
- Understand boolean type
- Build your own Java methods
- Define parameters and pass arguments to them
- Distinguish between class and instance methods
- Build a method library

Java's Math class

- Provides a set of math functions
- Part of Java.lang, so you don't have to import
- Static methods (no need to create a Math guy)
- **Math** methods most often take **double** arguments
- **Math** class constants: **Math.E** and **Math.PI**
- Example: compute volume of a sphere given radius
 - Formula: $\text{volume} = 4/3 \times \text{PI} \times \text{radius}^3$
 - Implementation: **double volume = 4.0 / 3.0 * Math.PI * Math.pow(radius, 3.0);**

Math Class

Math Method	Description
<code>Math.abs(v)</code>	Returns the absolute value of <code>v</code> , which can be a <code>double</code> , <code>float</code> , <code>int</code> , or <code>long</code>
<code>Math.acos(v)</code>	Returns the angle whose cosine is <code>v</code>
<code>Math.asin(v)</code>	Returns the angle whose sine is <code>v</code>
<code>Math.atan(v)</code>	Returns the angle whose tangent is <code>v</code>
<code>Math.atan2(y, x)</code>	Converts the rectangular coordinates <code>(x, y)</code> to polar coordinates <code>(r, theta)</code> , and returns the <code>theta</code> portion
<code>Math.cbrt(v)</code>	Returns the cube root of <code>v</code>
<code>Math.ceil(v)</code>	Returns the smallest <code>double</code> value that is an integer greater than or equal to <code>v</code>
<code>Math.cos(v)</code>	Returns the cosine of the angle <code>v</code>
<code>Math.cosh(v)</code>	Returns the hyperbolic cosine of <code>v</code>
<code>Math.exp(v)</code>	Returns <code>e</code> to the power <code>v</code> , where <code>e</code> is Euler's number
<code>Math.expml(v)</code>	Returns <code>e^{v-1}</code>
<code>Math.floor(v)</code>	Returns the largest <code>double</code> value that is an integer less than or equal to <code>v</code>
<code>Math.hypot(x, y)</code>	Returns the square root of $(x^2 + y^2)$
<code>Math.IEEEremainder(v1, v2)</code>	Returns the remainder of <code>v1/v2</code>
<code>Math.log(v)</code>	Returns the natural logarithm (base <code>e</code>) of <code>v</code>

continued

Math Class

Math Method	Description
<code>Math.log10(v)</code>	Returns the base-10 logarithm of <code>v</code>
<code>Math.log1p(v)</code>	Returns the natural logarithm of the value (<code>v+1</code>)
<code>Math.max(x, y)</code>	Returns the maximum of <code>x</code> and <code>y</code> , which can be <code>double</code> , <code>float</code> , <code>int</code> , or <code>long</code> values
<code>Math.min(x, y)</code>	Returns the minimum of <code>x</code> and <code>y</code> , which can be <code>double</code> , <code>float</code> , <code>int</code> , or <code>long</code> values
<code>Math.pow(x, y)</code>	Returns the value <code>x</code> to the power <code>y</code>
<code>Math.random()</code>	Returns a random value <code>x</code> ; <code>0.0 <= x</code> and <code>x < 1.0</code>
<code>Math rint(v)</code>	Returns a <code>double</code> value that is the integer closest to <code>v</code>
<code>Math.round(v)</code>	If <code>v</code> is a <code>double</code> , returns the <code>long</code> closest to <code>v</code> If <code>v</code> is a <code>float</code> , returns the <code>int</code> closest to <code>v</code>
<code>Math.signum(v)</code>	Returns <code>0.0</code> if <code>v</code> is zero; <code>1.0</code> if <code>v</code> is positive; and <code>-1.0</code> if <code>v</code> is negative. <code>v</code> may be a <code>double</code> or a <code>float</code>
<code>Math.sin(v)</code>	Returns the sine of angle <code>v</code>
<code>Math.sinh(v)</code>	Returns the hyperbolic sine of <code>v</code>
<code>Math.sqrt(v)</code>	Returns the rounded positive square root of <code>v</code>
<code>Math.tan(v)</code>	Returns the tangent of angle <code>v</code>
<code>Math.toDegrees(v)</code>	Returns the degrees equivalent to angle <code>v</code> (in radians)
<code>Math.toRadians(v)</code>	Returns the radians equivalent to angle <code>v</code> (in degrees)
<code>Math.ulp(v)</code>	Returns the “degenerate” units in the last place of <code>v</code> ; the distance between <code>v</code> and the next largest <code>double</code> value

FIGURE 8-5 Functions from Java’s `Math` class

The **String** Type

- Used to store a sequence of characters
- Example: **String** **lastName** = **"Cat"**;
- Different handles may refer to one **String** object
- **String** literal: 0 or more characters between “ and ”
 - Escape sequences can also be used in **String** literals
- **String** handle can be set to **null** or empty string

The **String** Type (continued)

- Instance method: message sent to an object
- Class method: message sent to a class
 - Indicated by the word **static**
- Java API lists a rich set of **String** operations
- Example of instance method sent to **String** object
 - **char lastInit = lastName.charAt(0);**
- Example of class method sent to **String** class
 - **String PI_STR = String.valueOf(Math.PI);**

The String Type (continued)

String Method	Returns
<code>str.charAt(i)</code>	the <code>char</code> located at index <code>i</code> within <code>str</code>
<code>str.endsWith(str2)</code>	<code>true</code> , if <code>str2</code> is the suffix of <code>str</code> ; <code>false</code> otherwise
<code>str.equals(str2)</code>	<code>true</code> , if <code>str2</code> and <code>str</code> contain the same characters considering case (<code>'A' != 'a'</code>); <code>false</code> otherwise
<code>str.equalsIgnoreCase(str2)</code>	<code>true</code> , if <code>str2</code> and <code>str</code> contain the same characters ignoring case (<code>'A' == 'a'</code>); <code>false</code> otherwise

continued

The String Type (continued)

String Method	Returns
<code>str.indexOf(ch)</code>	the index of the first occurrence of char <code>ch</code> within str ; -1 if not present
<code>str.indexOf(str2)</code>	the index at which the first occurrence of String <code>str2</code> begins within str ; -1 if not present
<code>str.lastIndexOf(ch)</code>	the index of the last occurrence of char <code>ch</code> within str ; -1 if not present
<code>str.lastIndexOf(str2)</code>	the index at which the last occurrence of str2 begins within str ; -1 if not present
<code>str.length()</code>	the number of characters in str
<code>str.startsWith(str2)</code>	true , if str2 is the prefix of str ; false otherwise
<code>str.substring(start, stop)</code>	a copy of the portion of str that begins at index start and that ends at index stop-1
<code>str.toLowerCase()</code>	a copy of str containing only lowercase characters
<code>str.toUpperCase()</code>	a copy of str containing only uppercase characters
<code>str.trim()</code>	a copy of str , minus any leading or trailing spaces
<code>String.valueOf(b)</code>	the String equivalent of boolean value b
<code>String.valueOf(c)</code>	the String equivalent of char value c
<code>String.valueOf(d)</code>	the String equivalent of double value d
<code>String.valueOf(i)</code>	the String equivalent of int value i
<code>String.valueOf(o)</code>	the String equivalent of the Object named o

FIGURE 8-12 Some of the `String` methods

The **String** Type (continued)

- Concatenation
 - Joins **String** values using the **+** operator
 - At least one operand must be a **String** type
- An illustration of concatenation
 - **String word = "good";**
word = word + "bye";
 - Second statement refers to a new object
 - Garbage collector disposes of the de-referenced object
- **+=**: the concatenation-assignment shortcut
 - Example: **word += "bye";**

The **boolean** Type

- Holds one of two values: **true** (1) or **false** (0)
- **boolean (logical)** expressions
 - Control flow of execution through a program
- Relational operators (<, >, <=, >=, ==, !=)
 - Compare two operands and return a **boolean** value
 - May also be used to build simple logical expressions
- Example of a simple **boolean** expression
 - **boolean seniorStatus = age >= 65;**
 - Produces **true** if **age >= 65**; otherwise **false**

The boolean Type (continued)

Relational Operator	Produces
<code>x == y</code>	<code>true</code> if <code>x</code> and <code>y</code> contain the same values; <code>false</code> otherwise.
<code>x != y</code>	<code>true</code> if <code>x</code> and <code>y</code> contain different values; <code>false</code> otherwise.

continued

The boolean Type (continued)

Relational Operator	Produces
<code>x < y</code>	<code>true</code> if <code>x</code> 's value is less than <code>y</code> 's value; <code>false</code> otherwise.
<code>x > y</code>	<code>true</code> if <code>x</code> 's value is greater than <code>y</code> 's value; <code>false</code> otherwise.
<code>x <= y</code>	<code>true</code> if <code>x</code> 's value is less than or equal to <code>y</code> 's value; <code>false</code> otherwise.
<code>x >= y</code>	<code>true</code> if <code>x</code> 's value is greater than or equal to <code>y</code> 's value; <code>false</code> otherwise.

FIGURE 8-7 The relational operators

The **boolean** Type (continued)

- Logical operators (**&&**, **||**, and **!**)
 - Used to build compound logical expressions
- Example of a compound logical expression
 - **boolean liqWater; // declare boolean variable**
liqWater = 0.0 < wTemp && wTemp < 100.0;
 - **wTemp** must be **> 0** and **< 100** to produce **true**
- Truth table
 - Relates combinations of operands to each operator
 - Shows value produced by each logical operation

The boolean Type (continued)

Logical Operator	Produces
<code>a && b</code>	<code>true</code> if <code>a</code> and <code>b</code> are both <code>true</code> ; <code>false</code> otherwise.
<code>a b</code>	<code>true</code> if <code>a</code> or <code>b</code> is <code>true</code> ; <code>false</code> otherwise.
<code>!b</code>	<code>true</code> if <code>b</code> is <code>false</code> ; <code>false</code> if <code>b</code> is <code>true</code> .

FIGURE 8-8 The logical operators

Methods

- How to perform a method
 - Send a message to an object or class
- Building a method in Alice
 - Click the **create new method** button
 - Drag statements into the method
- Focus of Chapter 9
 - Learning how to build methods in Java
- **You have been creating main methods, and might have created other methods also in the last homework**

Introductory Example: The Hokey Pokey Song

- Problem: write a Java program to display song lyrics
- Brute force approach
 - One **String** object stores the song lyrics
 - One action displays those lyrics
 - Implement program using one **println()** message
 - Issue: program is about 60 lines long (excessive)
- A better approach takes advantage of song structure
 - Each verse only differs by the body part that is moved
 - Implement program with a single method to print verse
 - **printVerse()** takes one argument for the **bodyPart**

Introductory Example: The Hokey Pokey Song (continued)

<i>You put your right foot in, you put your right foot out; you put your right foot in, and you shake it all about. You do the Hokey Pokey, and you turn yourself around. That's what it's all about!</i>	<i>You put your left foot in, you put your left foot out; you put your left foot in, and you shake it all about. You do the Hokey Pokey, and you turn yourself around. That's what it's all about!</i>
<i>You put your right hand in, you put your right hand out; you put your right hand in, and you shake it all about. You do the Hokey Pokey, and you turn yourself around. That's what it's all about!</i>	<i>You put your left hand in, you put your left hand out; you put your left hand in, and you shake it all about. You do the Hokey Pokey, and you turn yourself around. That's what it's all about!</i>
<i>You put your nose in, you put your nose out; you put your nose in, and you shake it all about. You do the Hokey Pokey, and you turn yourself around. That's what it's all about!</i>	<i>You put your backside in, you put your backside out; you put your backside in, and you shake it all about. You do the Hokey Pokey, and you turn yourself around. That's what it's all about!</i>
<i>You put your head in, you put your head out; you put your head in, and you shake it all about. You do the Hokey Pokey, and you turn yourself around. That's what it's all about!</i>	<i>You put your whole self in, you put your whole self out; you put your whole self in, and you shake it all about. You do the Hokey Pokey, and you turn yourself around. That's what it's all about!</i>

Introductory Example: The Hokey Pokey Song (continued)

```
1  /** HokeyPokey.java displays the lyrics of the Hokey Pokey song.
2      *      ...
3      */
4
5  public class HokeyPokey {
6      public static void main(String[] args) {
7          printVerse("right foot");
8          printVerse("left foot");
9          printVerse("right hand");
10         printVerse("left hand");
11         printVerse("nose");
12         printVerse("backside");
13         printVerse("head");
14         printVerse("whole self");
15     }
```

continued

Introductory Example: The Hokey Pokey Song (continued)

```
16
17 /** Display a verse of the Hokey Pokey song, given a body part.
18  * @param bodyPart, a String.
19  */
20 public static void printVerse(String bodyPart) {
21     System.out.println("You put your " + bodyPart + " in,\n"
22     + "you put your " + bodyPart + " out;\n"
23     + "you put your " + bodyPart + " in,\n"
24     + "and you shake it all about.\n"
25     + "You do the Hokey Pokey,\n"
26     + "and you turn yourself around.\n"
27     + "That's what it's all about!\n");
28 }
29 }
```

FIGURE 9-1 HokeyPokey.java

Methods (continued)

- Analyzing the first line of **printVerse()**
 - **public**: allows another class access to the method
 - **static**: indicates that the message is a class method
 - **void**: indicates that the method does not return a value
 - **printVerse**: the method's name
 - **()**: contains parameters, such as **String bodyPart**
 - **{**: indicates the beginning of the method statements
- Simplified pattern for a Java method
***[AccessMode] [static] ReturnType
MethodName (Params) {Statements}***

Method Design

- Procedure for developing a method
 - Figure out inputs and outputs (story) of the whole problem. (Test data here can help)
 - Figure out what repeats or is complex enough to splice out - > These are your methods
 - Maybe flow chart the main routine
 - Figure out the inputs and outputs for each method
 - List test data for the inputs and outputs
 - Determine Locals: variables and constants declared in a method

Exercise for Methods

Your task: Print a story that says:

I am a lonely cat, and I really like talking to you.

I am a sad cat, and I really like talking to you.

I am a mad cat, and I really like talking to you.

You came home!

I am a happy cat, and I really like talking to you.

Remember to figure out what your methods will be
and what your main program flow will be.

Non-**void** vs. **void** Methods

- Alice messages
 - Methods: just runs statements
 - Functions: returns a value
- Java - both are called methods
- **void** method in Java
 - Corresponds to an Alice method
 - Example: **printVerse()**
- non-**void** method in Java
 - Corresponds to an Alice function
 - Must have a return type

Einstein's Formula

- $e = m \times c^2$: energy = mass x speed of light²
 - The formula itself serves as the user story
 - Method returns an expression for right side of formula
- Developing the **massToEnergy()** method
 - Method's return type is a **double**
 - Parameter list includes a **double** type called **mass**
 - Speed of light is declared as a constant outside method
 - Computation is performed within **return** statement
- Example of a call to **massToEnergy()**
 - **double energy = massToEnergy(1.0);**

Einstein's Formula (continued)

The diagram illustrates the execution of the `massToEnergy()` method call. It features two code snippets. The top snippet shows a call to `double energy = massToEnergy(1.0);`, where the argument `1.0` is circled in red. A red arrow labeled 'A' points from this circled argument to the parameter `double mass` in the method signature of the bottom snippet. The bottom snippet shows the method definition: `public static double massToEnergy(double mass) { final double SPEED_OF_LIGHT = 2.99792458e8; return mass * Math.pow(SPEED_OF_LIGHT, 2.0); }`. A red arrow labeled 'B' points from the opening curly brace of the method body to the `return` statement. A red arrow labeled 'C' points from the `return` statement back to the `massToEnergy` method name in the top snippet. Finally, a red arrow labeled 'D' points from the `massToEnergy` method name in the top snippet to the `energy` variable.

```
double energy = massToEnergy(1.0);  
  
public static double massToEnergy(double mass) {  
    final double SPEED_OF_LIGHT = 2.99792458e8;  
    return mass * Math.pow(SPEED_OF_LIGHT, 2.0);  
}
```

FIGURE 9-4 Performing `massToEnergy()`

Non-void Method exercise

- Write a method called getBMI that calculates a person's Body Mass Index. The formula is: **BMI = Weight (lb) / (Height (in) x Height (in)) x 703**
- Example : Someone who is 5'6" (5'6" = 66") and weights 160 lb has a BMI of $160 / (66 \times 66) \times 703 = 25.8$

In your main program, print the following 2 lines:

At 66" and 160lb, Ted has a bmi of 25.8

At 55" and 160lb, Mary has a bmi of <whatever it returns>

Extra: Mary and Ted together have an average bmi of ??

How to start

- What goes in to the equation → your parameters
- What gets sent back → your return type
- What are the steps to get from one to the other → your statements

Method Tester

- Call your method from another class
- It can call the method many times sending it many different values.
- It should have one call for each of your test statements.
- It is just another class with a method called *testerNameTested* and statements calling the methods it tests.

Method Libraries

- Repositories for related methods
- Example: **Math** class
- Section objective: build two method libraries

Problem Description: Ballooning a Bedroom

- Problem context
 - Your friend who plays practical jokes is away
 - You want to play a practical joke on your friend
 - You plan to fill your friend's room with balloons
- Question: how many balloons should you purchase
- The question will be answered by a program

Program Design

- The problem is concerned with volumes
 - Find out how many balloon volumes fit in a room volume
- The balloon is approximated by a sphere
 - $\text{volume}_{\text{sphere}} = \frac{4}{3} \times \text{PI} \times \text{radius}^3$
- The room is approximated by a box
 - $\text{volume}_{\text{box}} = \text{length} \times \text{width} \times \text{height}$
- Another issue: whether to use large or small balloons
 - Large balloons take long to inflate, but fewer are needed
 - Small balloons inflate quickly, but more are needed

Program Design (continued)

- Essentials of the user story
 - Query the user for the radius of the balloon
 - Read the radius from the keyboard
 - Compute the volume of one balloon
 - Compute the volume of the bedroom
 - Note: dimensions of room are declared as constants
 - Compute number of balloons needed to fill the bedroom
 - Display the required number of balloons, with labels
- Identify nouns and verbs to find objects and operations
- Organize objects and operations into an algorithm

Program Design (continued)

Noun Phrase	Type of Value	Name
Our program	?	<i>BalloonPrank</i>
prompt	String	none
radius of a balloon	double	<i>balloonRadius</i>
the keyboard	Scanner	<i>keyboard</i>
volume of one balloon	double	<i>balloonVolume</i>
volume of the bedroom	double	<i>bedroomVolume</i>
required # of balloons	int	<i>balloonsRequired</i>

Program Design (continued)

Verb Phrase	Java Method or Operation
display ...	<code>System.out.print()</code> or <code>println()</code>
read radius	<code>balloonRadius = keyboard.nextDouble()</code>
compute volume of a balloon	<code>balloonVolume = ?</code>
compute volume of bedroom	<code>bedroomVolume = ?</code>
compute balloons required	<code>balloonsRequired = bedroomVolume / balloonVolume</code>

Program Design (continued)

Algorithm: BalloonPrank

1. Define constants *LENGTH*, *WIDTH*, *HEIGHT*
2. Display "Enter the radius of a balloon" via *System.out*
3. Read *balloonRadius* from keyboard
4. Compute *balloonVolume* from *balloonRadius*
5. Compute *bedroomVolume* from *LENGTH*, *WIDTH*, *HEIGHT*
6. Compute *balloonsRequired* = *bedroomVolume* / *balloonVolume*
7. Display *balloonRadius*, *balloonsRequired*, and descriptive labels.

FIGURE 9-10 Algorithm for the `BalloonPrank` program

Program Implementation

- First decision: write methods to compute volumes
 - Rationale: methods allow computations to be reused
- Second decision: store methods in separate classes
 - Rationale: makes the program more modular
- Three classes will be used to implement the program
 - **BalloonPrank**: contains the **main()** driver method
 - **Sphere**: library containing sphere methods
 - **Box**: library containing box methods
- **Sphere.volume()**: takes one argument (radius)
- **Box.volume()**: takes three arguments (l, w, h)

Program Implementation (continued)

```
1  /** BalloonPrank.java reads a balloon's radius
2   *   and displays the number of balloons needed to fill a bedroom.
3   */
4  import java.util.Scanner;
5
6  public class BalloonPrank {
7      public static void main(String[] args) {
8          final double LENGTH = 9.0;  // bedroom dimensions (in feet)
9          final double WIDTH  = 8.0;
10         final double HEIGHT = 7.0;
11         // get the balloon radius
12         System.out.print("Enter a balloon radius (inches): ");
13         Scanner keyboard = new Scanner(System.in);
14         double balloonRadius = keyboard.nextDouble();
15         balloonRadius /= 12.0;          // convert inches to feet
16         // compute the balloon and bedroom volumes
17         double balloonVolume = Sphere.volume(balloonRadius);
18         double bedroomVolume = Box.volume(LENGTH, WIDTH, HEIGHT);
19         // compute balloons in bedroom
20         double balloonsRequired = bedroomVolume / balloonVolume;
21         System.out.printf("Using balloons of radius %.0f inches,"
22                          + " you will need %.0f balloons",
```

continued

Program Implementation (continued)

```
23         balloonRadius*12, balloonsRequired);  
24     }  
25 }
```

Result:

Enter a balloon radius (inches): 6

Using balloons of radius 6 inches, you will need 963 balloons

Result:

Enter a balloon radius (inches): 4

Using balloons of radius 4 inches, you will need 3249 balloons

FIGURE 9-11 The `BalloonPrank.java` program

Program Implementation (continued)

```
1  /** Sphere.java provides sphere-related methods.
2   */
3  public class Sphere {
4      /** volume of a sphere (class method)
5       * @param radius, the (double) radius of the sphere
6       * @return the volume of a sphere with that radius
7       */
8      public static double volume(double radius) {
9          return 4.0 / 3.0 * Math.PI * Math.pow(radius, 3.0);
10     }
11     // ... other sphere-related methods omitted ...
12 }
13
```

FIGURE 9-12 A partial `sphere.java` class library

Unit Testing

- The sole purpose of a test class
 - Ensure that methods in the program or library work
- How to implement unit testing
 - Build a test class with test methods
 - One test method for each method in a program or library
 - Run the test methods
- Illustration of unit testing: **BoxTester.java**
 - Test method is named **testVolume()**
 - **testVolume()** tests the **volume()** method of **Box**
 - Note: test methods use Java's **assert** statement

Unit Testing (continued)

```
1  /** BoxTester.java tests the methods of Box.java.
2   */
3  public class BoxTester {
4      public static void main(String[] args) {
5          testVolume();
6          // add tests for other Box methods here
7      }
8
9      public static void testVolume() {
10         System.out.print("Testing Box.volume(): ");
11         double vol = Box.volume(2.0, 3.0, 4.0); // test 1
12         assert vol == 24.0;
13         System.out.print(" 1 ");
14         vol = Box.volume(6.0, 5.0, 4.0);          // test 2
15         assert vol == 120.0;
16         System.out.print(" 2 ");
17         System.out.println(" Passed!");
18     }
19 }
```

Result:

Testing Box.volume(): 1 2 Passed!

FIGURE 9-14 `BoxTester.java`

Test-Driven Development

- Reversing the normal testing process
 - Build the test (this is the starting point)
 - Use the test to drive subsequent method development
- Application to the development of methods
 - Method call indicates number of arguments needed
 - Number of arguments indicates number of parameters
 - Type of value expected indicates the return type
- Example: an initial test for **Box.volume()**
 - **double vol = Box.volume(2.0, 3.0, 4.0);**
assert vol == 24.0;

Instance Methods

- Method libraries do not use full capabilities of a class
 - Methods are used independently of objects
- Leveraging object-oriented programming features
 - Build objects with instance methods and variables
 - Send messages to objects
- Section objective
 - Learn how to define an instance method

Box Objects

- Disadvantage of **Box.volume()** (a class method)
 - Box dimensions are passed with each method call
- Alternative: call method against a **Box** object
 - Box initialized once, so values are passed only once
- Enabling **Box** class to become an object blueprint
 - Create instance variables for length, width, height
 - Names of **doubles**: **myLength**, **myWidth**, **myHeight**
 - Define accessor methods for the instance variables
 - Create a constructor for a **Box** object
 - Add an instance method for computing the volume

Box Objects (continued)



FIGURE 9-15 A `Box` handle and object

Box Objects (continued)

```
1  /** Box.java provides box-related methods.  
2   */  
3  public class Box {  
4      // instance variables to "remember" dimensions  
5      private double myLength, myWidth, myHeight;
```

continued

Box Objects (continued)

```
6
7 // Box constructor
8 public Box(double length, double width, double height) {
9     myLength = length;
10    myWidth = width;
11    myHeight = height;
12 }
13
14 // volume of a Box (instance method)
15 public double volume() { return myLength *
16                        myWidth * myHeight; }
17
18 // accessor methods for instance variables
19 public double getLength() { return myLength; }
20 public double getWidth() { return myWidth; }
21 public double getHeight() { return myHeight; }
22
23 // volume of a box (class method)
24 public static double volume(double length, double width,
25                             double height) {
26     return length * width * height;
27 }
28 // ... other methods omitted ...
29 }
```

FIGURE 9-16 The `Box` class, revised to support `Box` objects

Box Objects (continued)

- Characteristics of an instance variable
 - Defined within a class and outside of a method
 - Omits the keyword **static**
 - Each object has its own copy of the instance variables
- Characteristics of a class variable
 - Defined within a class and outside of a method
 - Includes the keyword **static**
 - All objects of a class share a class variable
- Access specifiers: **private**, **protected**, **public**
 - Guideline: use **private** access for instance variables

Box Objects (continued)

- Purpose of a constructor
 - Initialize instance variables with user-supplied values
- Constructor features
 - The constructor name is always the name of its class
 - A constructor has no return type (not even **void**)
- The **new** operator precedes a call to a constructor
 - Ex 1: **Box box1 = new Box(1.1, 2.2, 3.3);**
 - Ex 2: **Box box2 = new Box(9.9, 8.8, 7.7);**
- **box1** and **box2** contain references to **Box** objects

Box Objects (continued)

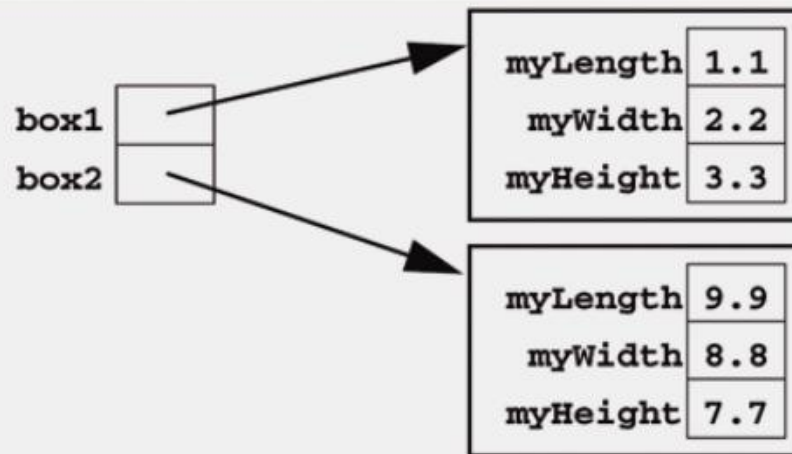


FIGURE 9-17 `Box` handles and objects

Box Objects (continued)

- Instance method
 - A message sent to an instance of a class
 - Not defined with the keyword **static**
 - Ex: **public double volume()**
{return myLength * myWidth * myHeight;}
- Invocation: **double box1Vol = box1.volume();**
- Accessor method (getter)
 - Instance method that returns value of instance variable
 - Name usually concatenates “get” with an attribute
 - Ex: **public double getWidth()**
{return myWidth;}

Sphere Objects

- Objective: enhance **Sphere** to support objects
- New members of **Sphere**
 - A single instance variable: **double** called **myRadius**
 - Instance method for calculating **Sphere** volume
 - An accessor to return the value of **myRadius**
- Sending messages to a **Sphere** object
 - **System.out.println(sphere1.volume());**
 - **System.out.println(sphere2.volume());**

Sphere Objects (continued)

```
1  /** Sphere.java provides sphere-related methods...
2   */
3  public class Sphere {
4      // instance variable
5      private double myRadius;
6
7      // constructor
8      public Sphere(double radius) { myRadius = radius; }
9
10     // volume of a Sphere (instance method)
11     public double volume() { return volume(myRadius); }
12
13     // accessor method for myRadius
14     public double getRadius() {return myRadius; }
15
16     // volume of a sphere (class method)
17     public static double volume(double radius) {
18         return 4.0 / 3.0 * Math.PI * Math.pow(radius, 3.0);
```

continued

Sphere Objects (continued)

```
19     }  
20     // ... other sphere-related methods omitted ...  
21 }
```

FIGURE 9-18 The `sphere` class, revised to support `sphere` objects

Sphere Objects (continued)

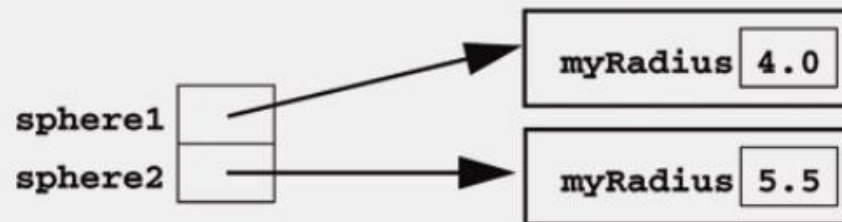


FIGURE 9-19 `sphere` handles and objects

The BalloonPrank Program Using Objects

- Program produces same results as the original
- Difference between original and enhanced versions
 - **Sphere** and **Box** objects model balloon and bedroom
- Chief benefit of the enhanced version
 - **Sphere** and **Box** classes can be used elsewhere
 - Ex: **Sphere earth = new Sphere(6356.75);**

The BalloonPrank Program Using Objects (continued)

```
1  /** BalloonPrank.java reads a balloon's radius and
2   *   displays the number of balloons needed to fill a bedroom.
3   */
4   import java.util.Scanner;
5
6   public class BalloonPrank {
7       public static void main(String[] args) {
8           // roommate's bedroom dimensions (in feet)
9           final double LENGTH = 9.0;
10          final double WIDTH = 8.0;
11          final double HEIGHT = 7.0;
12          // get the balloon radius
13          System.out.print("Enter a balloon radius (inches): ");
14          Scanner keyboard = new Scanner(System.in);
15          double balloonRadius = keyboard.nextDouble();
16          balloonRadius /= 12.0;           // convert inches to feet
17          // build balloon and bedroom models
18          Sphere balloon = new Sphere(balloonRadius);
19          Box bedroom = new Box(LENGTH, WIDTH, HEIGHT);
20          // compute balloons in bedroom
21          double balloonsRequired = bedroom.volume() /
                                   balloon.volume();
22          System.out.printf("Using balloons of radius %.0f inches,"
23                          + " you will need %.0f balloons",
24                          balloonRadius*12, balloonsRequired);
25      }
```

continued

The BalloonPrank Program Using Objects (continued)

26 }

Result:

Enter a balloon radius (inches): 6

Using balloons of radius 6 inches, you will need 963 balloons

Result:

Enter a balloon radius (inches): 4

Using balloons of radius 4 inches, you will need 3249 balloons

FIGURE 9-20 The revised `BalloonPrank` program

Classes, Methods, and Design

- Develop programs using procedure in Section 7.5
- Focus on second part of Step 2
 - To represent some objects, new types must be built
 - Ex: **Sphere** and **Box** types for balloon and bedroom
- Focus on the latter part of Step 3
 - If necessary, build a new method to perform an action
 - Ex: **volume()** methods built for **Sphere** and **Box**
- Abstraction:
 - Separating high-level behavior from low-level details
 - Methods and classes improve program abstraction

Classes, Methods, and Design (continued)

1. Write the user story. Identify the nouns and verbs within it.
2. For each noun in your algorithm: Find a type to represent that object. If no existing type will adequately represent the object, build a class to create a new type. Use that type to declare the object.
3. For each verb in your algorithm: Find a message that performs that verb's action, and send it to the verb's object. If no existing message that provides that verb's action, define a method to perform that action.
4. Organize your nouns and verbs into an algorithm that solves your problem.
5. Write your program, using your algorithm as a blueprint.
6. Test your program. If it produces incorrect results, return to Step 4.

Keywords, Identifiers, and Scope

- Keyword: word whose meaning is predefined
 - Examples: **class**, **int**, **void**, **static**, **double**
- Identifier: word whose meaning is user-defined
 - Declaration: provides identifier's meaning to compiler
 - Examples: **Box**, **Sphere**, **length**, **volume()**
- Scope: part of a program where an identifier is known
 - Scope for local identifiers: method's statement block
 - Scope for parameters: treated like local identifiers
 - Scope for class identifiers: the entire class block

Summary

- To make a group of statements reusable, place them within a method
- A class method includes the word **static** before the method's return type
- An instance method is sent to an object and does not include the word **static**
- A **void** method performs a set of actions, but returns no value
- A **non-void** method performs a set of actions, and returns a value

Summary (continued)

- Method library: class that serves as a repository for related methods
- Unit testing: a testing scheme that utilizes a test class containing a set of test methods
- Test-driven development: a testing scheme that uses desired test outcomes to drive method development
- Keywords, such as **static**, are predefined and identifiers, such as variable names, are user-defined
- Scope: portion of a program where an identifier has meaning